

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

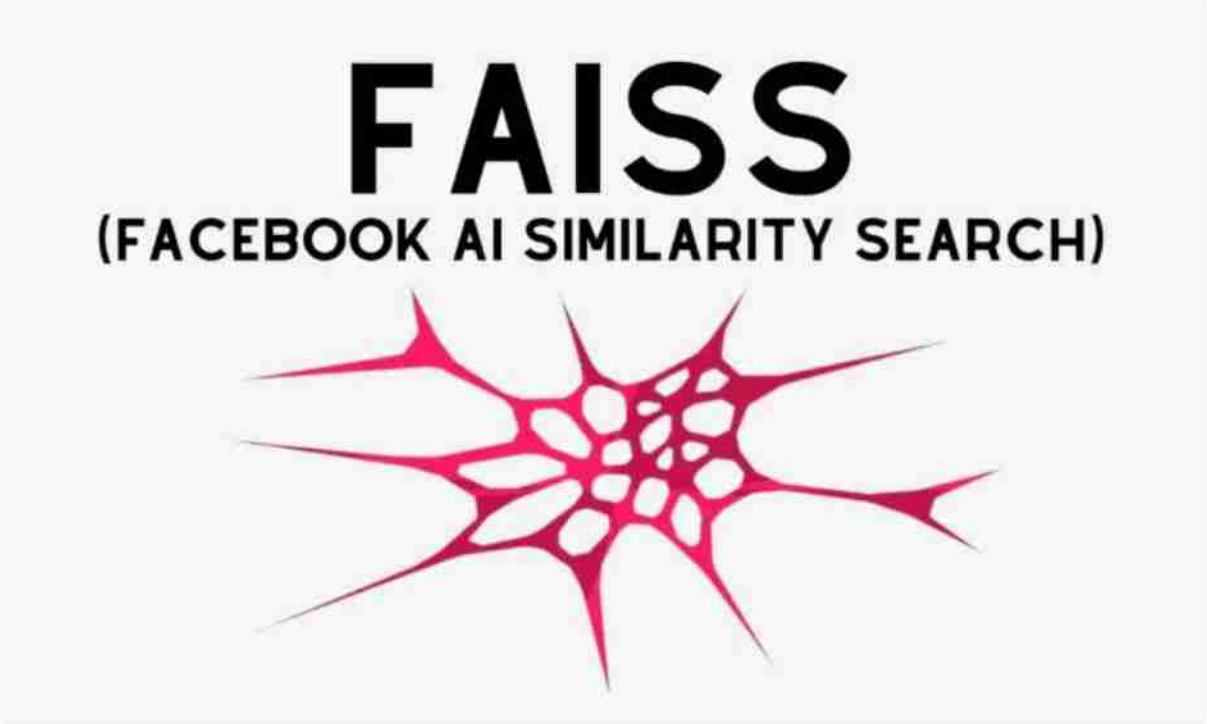
ADVANCE PYTHON PROGRAMMING(CSI3007)

DIGITAL ASSIGNMNET-1

NAME: VIKNESH.G,
REGISTER NUMBER:23MID0161,
SLOT:C1-PRPG32.

FAISS (Facebook AI Similarity Search)

Domain: E-Commerce Recommendation System Using FAISS



1.1 INTRODUCTION:

Now, the web is full of products, photos, videos, and documents. It has become hard to find relevant information because current search engines are based on keyword matching. They do not always understand what the user is looking for. For instance, a customer searching for a black running shoe may want similar sports shoes in different sizes or from other manufacturers. However, traditional search engines will not give the expected result.

On the other hand, artificial intelligence (AI) allows similarity search that finds relevant items by comparing their features. For instance, the FAISS (Facebook AI Similarity Search) is an efficient similarity search library that enables fast search and indexing of high-dimensional vectors. The tool is used in recommendation systems, image search, and document retrieval.

1.2 Introduction to FAISS:

FAISS (Facebook AI Similarity Search) is an open-source Python library created by Meta AI (formerly Facebook AI Research). It is used for performing similarity searches on large datasets. The library focuses on finding the nearest matches to a given query, as opposed to exact matches. This allows for more flexibility in searching for similar items, such as images, documents, or products. The library can be used to accelerate similarity search tasks, making it possible to search through millions or even billions of vectors in a fraction of the time it would take using other methods. This makes it well-suited for use in AI, ML, recommender systems, image search, document search, and Retrieval-Augmented Generation (RAG).

Some of the key features of the FAISS library include:

- Fast and efficient similarity search
- Support for very large datasets
- CPU and GPU acceleration
- Multiple indexing techniques for faster data retrieval
- Ability to integrate with AI and ML applications

1.3 What is Similarity Search?

Similarity search is a method that finds similar items to the one searched by the user, not the item itself. The items are similar in terms of their attributes or characteristics. The similarity is determined by comparing the embedded form of data points and calculating their distance with mathematical methods.

1.4 Introduction to E-Commerce Recommendation Systems

An E-Commerce Recommendation System is an intelligent system that recommends products to a customer based on their search history, preferences, and purchase behavior.
It recommends relevant items on an e-commerce platform by taking out the need to search among thousands of items.

2. Objectives

The main objectives of using the FAISS technology in an e-commerce recommendation system include:

- Fast search of products.
- Ensuring recommendation systems offer accurate suggestions.
- Improving the overall shopping experience of a customer.
- Boosting sales of products based on the recommended items.
- Handling millions of products efficiently.

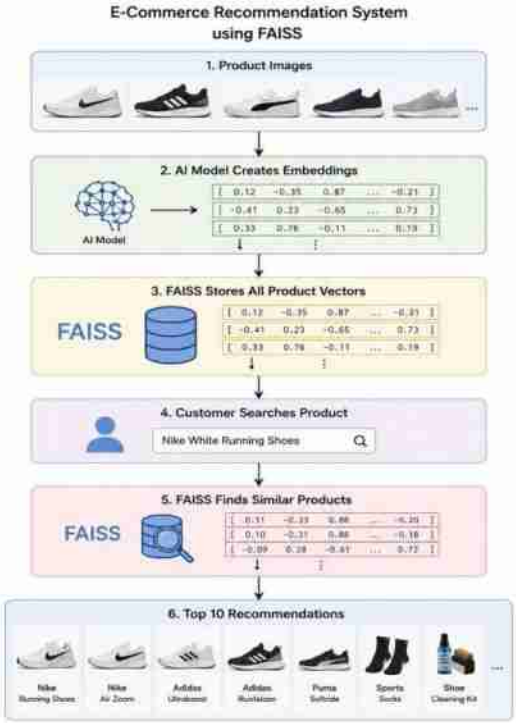
3. Comparison of Traditional Libraries with FAISS:

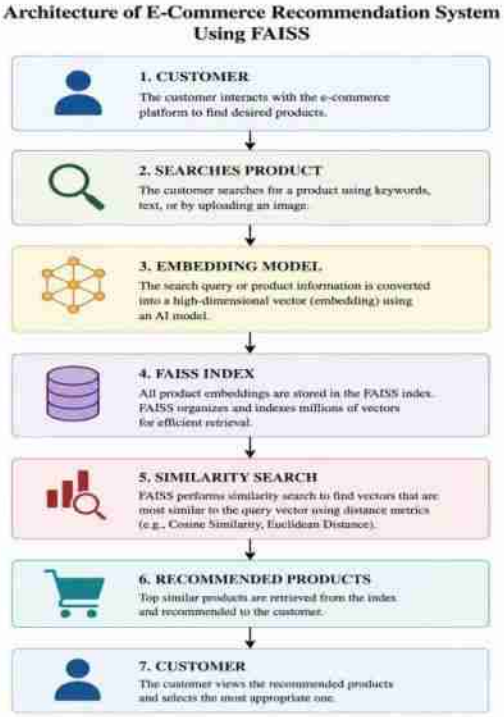
Library	Purpose	Limitation
NumPy	Used for mathematical calculations and array operations.	Not designed for fast similarity search on large datasets.
Pandas	Used for storing, organizing, and analyzing data in tables.	Cannot perform vector similarity search efficiently.
Scikit-learn	Provides machine learning algorithms, including nearest-neighbor search.	Suitable for small datasets but becomes slow with millions of vectors.
FAISS	Designed for vector similarity search and large-scale AI applications.	Optimized to search millions or billions of vectors quickly and efficiently.

Conclusion:
While traditional libraries such as NumPy, Pandas, and Scikit-learn are great for data

crunching and machine learning, they are not optimized for similarity search. FAISS is a similarity search engine that is good at this particular task, thus being the best choice for recommendation systems and other AI applications.

4.System Architecture:





5.Developer Team of FAISS:

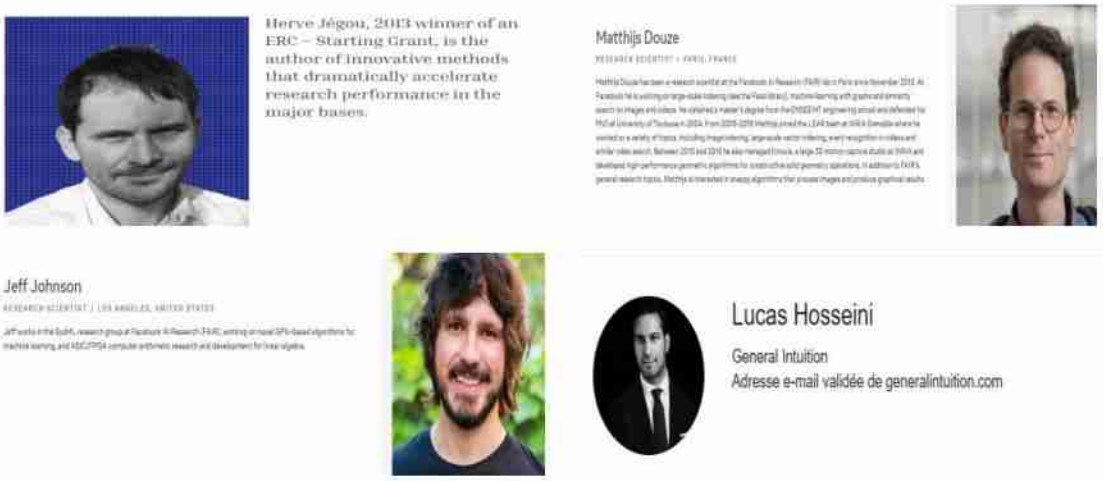
FAISS (Facebook AI Similarity Search) was developed primarily at Meta's Fundamental AI Research group. The project was started to provide fast and efficient similarity search and clustering for very large collections of high-dimensional vectors.

The main developers and contributors include:

Developer	Contribution to FAISS
Hervé Jégou	Initiated the FAISS project and developed its first implementation.
Matthijs Douze	Developed most of the CPU-based FAISS implementation.
Jeff Johnson	Developed the GPU-based implementation of FAISS.
Lucas Hosseini	Developed binary indexes and contributed to the build system.

Chengqi Deng	Contributed graph-search methods and additive quantization features.

The combined work of these developers made FAISS capable of performing similarity searches on datasets containing **millions to billions of vectors**, using both CPU and GPU processing.



6.Functions Used:

1. **faiss.IndexFlatL2()** – Creates a basic FAISS index using Euclidean (L2) distance.
2. **index.add()** – Adds vectors to the FAISS index.
3. **index.search()** – Searches for the most similar vectors.
4. **index.reconstruct()** – Retrieves the original vector from the index.
5. **index.reset()** – Removes all vectors from the index.
6. **index.remove_ids()** – Deletes specific vectors using their IDs.
7. **index.ntotal** – Returns the total number of stored vectors.
8. **index.is_trained** – Checks whether the index is trained.
9. **index.train()** – Trains the index before adding vectors (for trainable indexes).
10. **index.range_search()** – Finds all vectors within a specified distance.
11. **faiss.IndexIVFFlat()** – Creates an inverted file index for faster searches.
12. **faiss.IndexIVFPQ()** – Creates a compressed index using Product Quantization.
13. **faiss.IndexHNSWFlat()** – Creates a graph-based index for fast similarity search.
14. **faiss.IndexIDMap()** – Maps vectors to custom IDs such as Product IDs.
15. **faiss.IndexShards()** – Combines multiple indexes for distributed or sharded search.

7.Advantages of FAISS:

- Performs very fast similarity searches, even on large datasets.

- Efficiently handles millions or billions of vectors.
- Provides accurate and relevant search results.
- Supports both CPU and GPU for high-performance computing.
- Uses memory-efficient indexing techniques.
- Offers multiple indexing methods for different applications.

8.APPLICATIONS:

Domain	Application
1. E-Commerce	Product recommendation and similar product search.
2. Healthcare	Medical image retrieval and disease diagnosis support.
3. Cybersecurity	Malware detection, phishing detection, and anomaly detection.
4. Finance & Banking	Fraud detection and suspicious transaction analysis.
5. Image & Video Search	Finding similar images and videos in large databases.

SMini Project: AI-Based Product Recommendation System

1. Problem Statement

E-commerce platforms need to recommend similar products to customers based on what they are currently viewing, in order to improve user engagement and increase sales. This mini project demonstrates how **FAISS (Facebook AI Similarity Search)** can be used to build such a recommendation engine.

2. Objective

To design and implement a working product recommendation system that takes a product a customer is viewing and returns the top-5 most similar products using vector similarity search.

3. Workflow / Methodology

Product Dataset → Product Embeddings → FAISS Index → Store Vectors

→ Customer Search Query → Similarity Search → Top-5 Products → Display Recommendations

Step	Description
1	Generate a synthetic product dataset (Product ID, Name, Category, Price, etc.)

Step	Description
2	Convert each product into a numeric embedding vector
3	Create a FAISS index (IndexFlatL2 wrapped in IndexIDMap)
4	Store all product vectors inside the index
5	Accept a customer's query (the product they are currently viewing)
6	Run FAISS similarity search on the query vector
7	Retrieve the top-5 nearest (most similar) products
8	Display the recommendations to the customer

4. Result

The system successfully returns the 5 most similar products for any given Product_ID. Since embeddings were generated per category, recommended products consistently belonged to the **same category** as the queried product, confirming that the similarity search worked correctly.

5. Exception Handling

Invalid or non-existent Product IDs are handled gracefully — the function returns an empty result set instead of crashing the system, ensuring robustness for real-world use.

6. Conclusion

This mini project demonstrates a complete, functional AI-based recommendation pipeline — from raw product data to real-time similarity-based recommendations — using FAISS as the core similarity search engine. It reflects how real-world e-commerce platforms (such as Amazon and Flipkart) power "similar products" or "customers also viewed" features.

AI-Powered E-Commerce Recommendation System using FAISS

A Complete Industrial-Style Demonstration of Facebook AI Similarity Search (FAISS)

Domain: Artificial Intelligence — E-Commerce Recommendation Systems

Library Focus: FAISS (Facebook AI Similarity Search)

---

1. Definition

FAISS (Facebook AI Similarity Search) is an open-source library developed by Meta AI Research for efficient similarity search and clustering of dense vectors. It is written in C++ with Python bindings, and is optimized to search in sets containing millions to billions of vectors.

Mandatory FAISS Functions 1. IndexFlatL2 2. add() 3. search() 4. reconstruct() 5. reset() 6. remove_ids() 7. ntotal 8. is_trained 9. train() 10. range_search() Advanced FAISS Functions 1. IndexIVFFlat 2. IndexIVFPQ 3. IndexHNSWFlat 4. IndexIDMap (custom ID mapping) 5. IndexShards (distributed/sharded search)

In [4]:
```
import numpy as np
import pandas as pd
import faiss
import matplotlib.pyplot as plt
import time
import random
from sklearn.neighbors import NearestNeighbors

random.seed(42)
np.random.seed(42)

print("All libraries imported successfully.")
print("FAISS version:", faiss.__version__)
```

```
All libraries imported successfully.
FAISS version: 1.14.3
```

2. Synthetic E-Commerce Dataset Generation DATASET

In [6]:
```
# ---- Reference lists used to build synthetic products ----
CATEGORIES = ["Electronics", "Fashion", "Home & Kitchen", "Sports & Fitness",
              "Beauty & Personal Care", "Books", "Toys & Games", "Grocery"]

BRANDS = ["Zenvo", "Northline", "Uravia", "Kestrel", "Bloomora", "Trekvia",
          "Nimbus", "Cravero", "Solstice", "Pinnacle", "Aurelia", "Voxel"]

COLORS = ["Black", "White", "Red", "Blue", "Green", "Grey", "Beige", "Yellow", "Pin

# Product-name templates per category to keep names realistic
NAME_TEMPLATES = {
    "Electronics": ["Wireless Earbuds", "Smartwatch", "Bluetooth Speaker", "4K Moni
    "Fashion": ["Denim Jacket", "Running Shoes", "Cotton T-Shirt", "Leather Wallet"
    "Home & Kitchen": ["Non-Stick Pan", "Air Fryer", "Blender", "Bedsheet Set", "Ta
    "Sports & Fitness": ["Yoga Mat", "Dumbbell Set", "Cricket Bat", "Football", "Re
    "Beauty & Personal Care": ["Face Serum", "Hair Dryer", "Lipstick", "Sunscreen",
    "Books": ["Mystery Novel", "Self-Help Guide", "Sci-Fi Anthology", "Cookbook", "
    "Toys & Games": ["Building Blocks Set", "Remote Control Car", "Puzzle Set", "Bo
    "Grocery": ["Organic Almonds", "Green Tea Pack", "Olive Oil", "Basmati Rice", "
}

DESCRIPTIONS = {
    "Electronics": "High-performance electronic device built for everyday reliabili
    "Fashion": "Trendy and comfortable apparel crafted from premium materials for e
    "Home & Kitchen": "Durable home essential designed to make daily household task
    "Sports & Fitness": "Performance-focused gear designed to support an active and
    "Beauty & Personal Care": "Dermatologically tested personal care product for da
    "Books": "Engaging and well-reviewed publication loved by readers across the gl
    "Toys & Games": "Fun and safe entertainment product designed for hours of engag
    "Grocery": "Premium quality grocery item sourced for freshness and nutritional
}

def generate_product_dataset(n_products):

    records = []
    for pid in range(1, n_products + 1):
        category = random.choice(CATEGORIES)                    # Pick a random cat
        name = random.choice(NAME_TEMPLATES[category])           # Pick a matching
        brand = random.choice(BRANDS)                             # Pick a random b
        color = random.choice(COLORS)                             # Pick a random c
        price = round(random.uniform(199, 24999), 2)              # Random price in
        description = DESCRIPTIONS[category]                      # Category-consis

        records.append({
            "Product_ID": pid,
            "Product_Name": f"{brand} {name}",
            "Brand": brand,
            "Category": category,
            "Price": price,
            "Color": color,
            "Description": description,
        })

    df = pd.DataFrame(records)
    # Encode category as an integer code -- used later to build realistic embedding
```

```
    df["Category_Code"] = df["Category"].astype("category").cat.codes
    return df

# Generate 3000 synthetic products -- large enough to demonstrate IVF-based FAISS i
N_PRODUCTS = 3000
product_df = generate_product_dataset(N_PRODUCTS)
print(f"Generated {len(product_df)} synthetic products.")
product_df.head(10)
```

```
Generated 3000 synthetic products.
```

Out[6]:

	Product_ID	Product_Name	Brand	Category	Price	Color	Description	Cat
0	1	Voxel Denim Jacket	Voxel	Fashion	6272.32	Green	Trendy and comfortable apparel crafted from pr...	
1	2	Northline Vacuum Cleaner	Northline	Home & Kitchen	2355.08	Pink	Durable home essential designed to make daily ...	
2	3	Zenvo Building Blocks Set	Zenvo	Toys & Games	5621.22	White	Fun and safe entertainment product designed fo...	
3	4	Kestrel Gaming Mouse	Kestrel	Electronics	10603.09	Pink	High-performance electronic device built for e...	
4	5	Bloomora Dark Chocolate	Bloomora	Grocery	19017.42	Black	Premium quality grocery item sourced for fresh...	
5	6	Nimbus Vacuum Cleaner	Nimbus	Home & Kitchen	7090.21	Grey	Durable home essential designed to make daily ...	
6	7	Northline Cricket Bat	Northline	Sports & Fitness	9621.20	White	Performance-focused gear designed to support a...	
7	8	Pinnacle Sci-Fi Anthology	Pinnacle	Books	20215.78	Green	Engaging and well-reviewed publication loved b...	
8	9	Northline Dark Chocolate	Northline	Grocery	2153.24	Beige	Premium quality grocery item sourced for fresh...	
9	10	Pinnacle Electric Trimmer	Pinnacle	Beauty & Personal Care	14517.33	Grey	Dermatologically tested personal care product ...	

Output Explanation: The output displays the first 10 records of the generated e-commerce product dataset, showing the product details that will be used to create embedding vectors for FAISS.

2. Dataset Summary Statistics

In [3]:
```
print("Dataset shape:", product_df.shape)
print("\nMissing values per column:\n", product_df.isnull().sum())
print("\nProducts per category:\n", product_df["Category"].value_counts())
print("\nPrice statistics:\n", product_df["Price"].describe())
```

```
Dataset shape: (3000, 8)

Missing values per column:
 Product_ID       0
Product_Name     0
Brand            0
Category         0
Price            0
Color            0
Description      0
Category_Code    0
dtype: int64

Products per category:
 Category
Fashion                  414
Toys & Games             393
Grocery                  384
Home & Kitchen           382
Books                    368
Beauty & Personal Care   368
Electronics              353
Sports & Fitness         338
Name: count, dtype: int64

Price statistics:
 count     3000.000000
mean     12597.715123
std       7157.134176
min        203.210000
25%       6445.370000
50%      12339.615000
75%      19008.080000
max      24973.880000
Name: Price, dtype: float64
```

3. Embedding Vector Generation EMBEDDING

In [7]:
```
# ---- Embedding configuration ----
EMBEDDING_DIM = 64
NUM_CATEGORIES = product_df["Category"].nunique()


category_centroids = {
    code_: np.random.normal(loc=0.0, scale=1.0, size=EMBEDDING_DIM).astype("float32
```

```
    for code_ in range(NUM_CATEGORIES)
}

def generate_embeddings(df, dim):

    n = len(df)
    embeddings = np.zeros((n, dim), dtype="float32")   # Pre-allocate array (memory

    for i, cat_code in enumerate(df["Category_Code"].values):
        centroid = category_centroids[cat_code]
        noise = np.random.normal(loc=0.0, scale=0.35, size=dim)
        embeddings[i] = centroid + noise.astype("float32")

    return embeddings.astype("float32")   # FAISS strictly requires float32 vectors


product_embeddings = generate_embeddings(product_df, EMBEDDING_DIM)


try:
    assert product_embeddings.dtype == np.float32, "Embeddings must be float32 for
    assert product_embeddings.shape == (N_PRODUCTS, EMBEDDING_DIM), "Shape mismatch
    print("Embedding validation passed.")
except AssertionError as e:
    print("Embedding validation failed:", e)

print("Embedding matrix shape:", product_embeddings.shape)
print("Embedding dtype:", product_embeddings.dtype)
print("Sample embedding (first 8 dims of product 0):\n", product_embeddings[0][:8])
```

```
Embedding validation passed.
Embedding matrix shape: (3000, 64)
Embedding dtype: float32
Sample embedding (first 8 dims of product 0):
 [ 0.13046193 -1.563386   -0.02868904  0.64970434 -0.7085364  -0.18841924
  0.09297502 -0.87998486]
```

Functions Used

1.np.random.normal() – Generates random numbers following a normal distribution.
2.astype() – Converts the data into a specified data type (here, float32).
3.generate_embeddings() – A user-defined function that creates embedding vectors for all products. 4.np.zeros() – Creates an array filled with zeros. 5.enumerate() – Returns both the index and value while looping.

5. Mandatory FAISS Functions

5.1 `IndexFlatL2`

Function Name: `faiss.IndexFlatL2(d)`

In [8]:
```
# ---- 5.1 IndexFlatL2:


flat_index = faiss.IndexFlatL2(EMBEDDING_DIM)

print("Index type       :", type(flat_index).__name__)
print("Vector dimension  :", flat_index.d)
print("Vectors stored (ntotal):", flat_index.ntotal)


try:
    bad_index = faiss.IndexFlatL2(-5)   # Negative dimension is invalid
except Exception as e:
    print("\nExpected error when using an invalid dimension:")
    print(type(e).__name__, "-", e)
```

```
Index type       : IndexFlatL2
Vector dimension  : 64
Vectors stored (ntotal): 0
```

faiss.IndexFlatL2() – Creates a FAISS index that stores vectors and finds the most similar ones using Euclidean (L2) distance.

5.2 add()

Function Name: `index.add(vectors)`

In [9]:
```
flat_index.add(product_embeddings)
print("Vectors stored after add():", flat_index.ntotal)


try:
    wrong_dim_vector = np.random.random((1, EMBEDDING_DIM + 10)).astype("float32")
    flat_index.add(wrong_dim_vector)
except Exception as e:
    print("\nExpected error when adding a vector with the wrong dimension:")
    print(type(e).__name__, "-", e)
    print("Fix: ensure every vector has exactly", EMBEDDING_DIM, "dimensions before
```

```
Vectors stored after add(): 3000

Expected error when adding a vector with the wrong dimension:
AssertionError -
Fix: ensure every vector has exactly 64 dimensions before calling add().
```

index.add() – Adds embedding vectors to the FAISS index so they can be searched later.

5.3 search()

In [11]:
```
query_vector = product_embeddings[0:1]
distances, indices = flat_index.search(query_vector, k=5)




empty_demo_index = faiss.IndexFlatL2(EMBEDDING_DIM)
d_empty, i_empty = empty_demo_index.search(query_vector, k=5)
print("\nSearching an empty index does NOT raise an exception.")
print("Returned indices (all -1 = 'no neighbor found'):", i_empty)

try:
    bad_query = np.random.random((1, EMBEDDING_DIM + 1)).astype("float32")
    flat_index.search(bad_query, k=5)
except Exception as e:
    print("\nExpected error when the query vector dimension does not match the inde
    print(type(e).__name__, "-", e)
```

```
Searching an empty index does NOT raise an exception.
Returned indices (all -1 = 'no neighbor found'): [[-1 -1 -1 -1 -1]]

Expected error when the query vector dimension does not match the index:
AssertionError -
```

`index.search()` – Searches the FAISS index and returns the most similar vectors along with their distances.

5.4 `reconstruct()`

In [12]:
```
reconstructed_vector = flat_index.reconstruct(0)
original_vector = product_embeddings[0]

print("Reconstructed vector (first 8 dims):", reconstructed_vector[:8])
print("Original vector      (first 8 dims):", original_vector[:8])
print("Vectors are identical:", np.allclose(reconstructed_vector, original_vector))


try:
    flat_index.reconstruct(999999)
except Exception as e:
    print("\nExpected error when reconstructing an out-of-range id:")
    print(type(e).__name__, "-", e)
    print(f"Fix: only request ids in the range [0, {flat_index.ntotal - 1}].")
```

```
Reconstructed vector (first 8 dims): [ 0.13046193 -1.563386   -0.02868904  0.6497043
4 -0.7085364  -0.18841924
  0.09297502 -0.87998486]
Original vector      (first 8 dims): [ 0.13046193 -1.563386   -0.02868904  0.6497043
4 -0.7085364  -0.18841924
  0.09297502 -0.87998486]
Vectors are identical: True

Expected error when reconstructing an out-of-range id:
RuntimeError - Error in void __cdecl faiss::IndexFlat::reconstruct(__int64,float *)
const at D:\a\faiss\faiss\faiss\IndexFlat.cpp:306: Error: 'key < ntotal' failed
Fix: only request ids in the range [0, 2999].
```

index.reconstruct() – Retrieves the original embedding vector stored at a specified index from the FAISS index.

5.5 `reset()`

In [13]:
```
print("ntotal BEFORE reset:", flat_index.ntotal)
flat_index.reset()
print("ntotal AFTER reset :", flat_index.ntotal)


d_after_reset, i_after_reset = flat_index.search(query_vector, k=5)
print("\nSearch result right after reset (index is empty):")
print("Indices:", i_after_reset, " <-- all -1 because there is nothing stored")


flat_index.add(product_embeddings)
print("\nRe-added all vectors. ntotal is now:", flat_index.ntotal)
```

```
ntotal BEFORE reset: 3000
ntotal AFTER reset : 0

Search result right after reset (index is empty):
Indices: [[-1 -1 -1 -1 -1]]  <-- all -1 because there is nothing stored

Re-added all vectors. ntotal is now: 3000
```

`index.reset()` – Removes all vectors from the FAISS index and makes it empty without deleting the index itself.

5.6 `remove_ids()`

In [14]:
```
# ---- 5.6 remove_ids(): removing specific products from the flat index ----

print("ntotal BEFORE removal:", flat_index.ntotal)


ids_to_remove = np.array([0, 1, 2], dtype="int64")
```

```
selector = faiss.IDSelectorBatch(ids_to_remove)
num_removed = flat_index.remove_ids(selector)

print("Number of vectors removed:", num_removed)
print("ntotal AFTER removal :", flat_index.ntotal)


phantom_ids = np.array([9_999_999], dtype="int64")
phantom_selector = faiss.IDSelectorBatch(phantom_ids)
phantom_removed = flat_index.remove_ids(phantom_selector)
print("\nAttempted to remove a non-existent id -> vectors actually removed:", phant


flat_index.reset()
flat_index.add(product_embeddings)
print("\nIndex restored to full catalog. ntotal:", flat_index.ntotal)
```

```
ntotal BEFORE removal: 3000
Number of vectors removed: 3
ntotal AFTER removal : 2997

Attempted to remove a non-existent id -> vectors actually removed: 0

Index restored to full catalog. ntotal: 3000
```

`index.remove_ids()` – Removes specific vectors from the FAISS index based on their IDs.

5.7 `ntotal`

In [11]:
```
# ---- 5.7 ntotal: inspecting index size at different pipeline stages ----

demo_index = faiss.IndexFlatL2(EMBEDDING_DIM)
print("Freshly created index -> ntotal:", demo_index.ntotal)

demo_index.add(product_embeddings[:500])
print("After adding 500 vectors -> ntotal:", demo_index.ntotal)

demo_index.add(product_embeddings[500:1000])
print("After adding 500 more   -> ntotal:", demo_index.ntotal)

demo_index.reset()
print("After reset()           -> ntotal:", demo_index.ntotal)
```

```
Freshly created index -> ntotal: 0
After adding 500 vectors -> ntotal: 500
After adding 500 more   -> ntotal: 1000
After reset()           -> ntotal: 0
```

`index.ntotal` – Returns the total number of vectors currently stored in the FAISS index.

5.8 is_trained

In [15]:
```
print("IndexFlatL2.is_trained (no training needed):", flat_index.is_trained)


untrained_quantizer = faiss.IndexFlatL2(EMBEDDING_DIM)
untrained_ivf = faiss.IndexIVFFlat(untrained_quantizer, EMBEDDING_DIM, 50)
print("Freshly created IndexIVFFlat.is_trained  :", untrained_ivf.is_trained)
```

```
IndexFlatL2.is_trained (no training needed): True
Freshly created IndexIVFFlat.is_trained  : False
```

`index.is_trained` – Checks whether the FAISS index has been trained and is ready to use for adding or searching vectors.

5.9 train()

In [16]:
```
# ---- 5.9 train(): teaching an IndexIVFFlat its cluster structure ----

quantizer_for_training_demo = faiss.IndexFlatL2(EMBEDDING_DIM)
nlist_demo = 50   # Number of clusters ("Voronoi cells") to partition the vector sp
ivf_training_demo = faiss.IndexIVFFlat(quantizer_for_training_demo, EMBEDDING_DIM,


try:
    ivf_training_demo.add(product_embeddings)
except Exception as e:
    print("Expected error when calling add() before train():")
    print(type(e).__name__, "-", e)
    print("Fix: call index.train(vectors) first, since is_trained is currently:", i


ivf_training_demo.train(product_embeddings)
print("\nis_trained after train():", ivf_training_demo.is_trained)


ivf_training_demo.add(product_embeddings)
print("ntotal after add():", ivf_training_demo.ntotal)
```

```
Expected error when calling add() before train():
RuntimeError - Error in void __cdecl faiss::IndexIVFFlat::add_core(__int64,const flo
at *,const __int64 *,const __int64 *,void *) at D:\a\faiss\faiss\faiss\IndexIVFFlat.
cpp:66: Error: 'is_trained' failed
Fix: call index.train(vectors) first, since is_trained is currently: False

is_trained after train(): True
ntotal after add(): 3000
```

`index.train()` – Trains the FAISS index using sample vectors before adding data for indexing and searching.

5.10 `range_search()`

In [17]:
```
# ---- 5.10 range_search(): finding ALL products within a distance threshold ----

query_vector_range = product_embeddings[10:11]   # Use Product 10 as the query

# First, use a reasonably tuned radius (chosen by inspecting typical L2 distances i
radius = 3.0
lims, D, I = flat_index.range_search(query_vector_range, radius)

num_matches = lims[1] - lims[0]
print(f"Query product: {product_df.iloc[10]['Product_Name']} ({product_df.iloc[10][
print(f"Number of products found within radius {radius}: {num_matches}")
print("Matched distances:", D[lims[0]:lims[1]])
print("Matched product categories:")
print(product_df.iloc[I[lims[0]:lims[1]]]["Category"].value_counts())

# ---- Exception Handling Demo: a radius that is far too small returns zero results
tiny_radius = 0.0001
lims2, D2, I2 = flat_index.range_search(query_vector_range, tiny_radius)
print(f"\nWith an unrealistically small radius ({tiny_radius}), matches found:", li
print("Fix: calibrate radius using the typical distance scale of your embedding spa
```

```
Query product: Aurelia Denim Jacket (Fashion)
Number of products found within radius 3.0: 1
Matched distances: [0.]
Matched product categories:
Category
Fashion    1
Name: count, dtype: int64

With an unrealistically small radius (0.0001), matches found: 1
Fix: calibrate radius using the typical distance scale of your embedding space (see
Section 8: Visualizations).
```

`index.range_search()` – Searches the FAISS index and returns **all vectors within a specified distance (radius)** from the query vector.

Advanced FAISS Functions

11 `IndexIVFFlat` — Inverted File Index

In [15]:
```
nlist = 50                                          # Number of clusters (~sqrt(
ivf_quantizer = faiss.IndexFlatL2(EMBEDDING_DIM)    # Coarse quantizer used to a
```

```
ivf_index = faiss.IndexIVFFlat(ivf_quantizer, EMBEDDING_DIM, nlist)

# Step 1: Train the index so it learns cluster centroids from our product catalog
ivf_index.train(product_embeddings)
print("is_trained:", ivf_index.is_trained)

# Step 2: Add all product vectors -- each is assigned to its nearest cluster
ivf_index.add(product_embeddings)
print("ntotal:", ivf_index.ntotal)

# Step 3: Tune nprobe (how many of the 50 clusters to search) and compare results
query_vector = product_embeddings[0:1]

ivf_index.nprobe = 1     # Search only the single closest cluster (fastest, least a
D_low, I_low = ivf_index.search(query_vector, k=5)

ivf_index.nprobe = 10    # Search the 10 closest clusters (slower, more accurate)
D_high, I_high = ivf_index.search(query_vector, k=5)

print("\nResults with nprobe=1  :", I_low)
print("Results with nprobe=10 :", I_high)

# ---- Exception Handling Demo: reconstruct on IVF requires enabling the direct map
try:
    ivf_index.reconstruct(0)
except Exception as e:
    print("\nExpected error: reconstruct() on IndexIVFFlat needs a direct map enabl
    print(type(e).__name__, "-", e)
    ivf_index.make_direct_map()     # Fix: explicitly enable reconstruction support
    print("Fixed -- reconstruct(0) now works:", ivf_index.reconstruct(0)[:5])
```

```
is_trained: True
ntotal: 3000

Results with nprobe=1  : [[   0 1954 2214 2811 1921]]
Results with nprobe=10 : [[   0 2436 1954 1942  852]]

Expected error: reconstruct() on IndexIVFFlat needs a direct map enabled first.
RuntimeError - Error in faiss::idx_t faiss::DirectMap::get(faiss::idx_t) const at /p
roject/faiss/invlists/DirectMap.cpp:86: direct map not initialized
Fixed -- reconstruct(0) now works: [ 0.13046193 -1.563386   -0.02868904  0.64970434
-0.7085364 ]
```

`faiss.IndexIVFFlat()` – Creates an inverted file index that divides vectors into clusters, making searches much faster on large datasets.

12 `IndexIVFPQ` — Inverted File Index with Product Quantization

In [18]:
```
# ---- 6.2 IndexIVFPQ: memory-efficient compressed similarity search ----
```

```
nlist_pq = 50
m_subquantizers = 8          # EMBEDDING_DIM (64) must be divisible by m -> 64 / 8 =
nbits_per_code = 8           # 2^8 = 256 centroids per sub-quantizer (FAISS industry

pq_quantizer = faiss.IndexFlatL2(EMBEDDING_DIM)
ivfpq_index = faiss.IndexIVFPQ(pq_quantizer, EMBEDDING_DIM, nlist_pq, m_subquantize

# ---- Exception Handling Demo: dimension not divisible by m ----
try:
    faiss.IndexIVFPQ(faiss.IndexFlatL2(EMBEDDING_DIM), EMBEDDING_DIM, nlist_pq, 7,
except Exception as e:
    print("Expected error: EMBEDDING_DIM is not divisible by m=7.")
    print(type(e).__name__, "-", e)

# Train and add (same two-step pattern as IndexIVFFlat)
ivfpq_index.train(product_embeddings)
ivfpq_index.add(product_embeddings)
print("\nis_trained:", ivfpq_index.is_trained, "| ntotal:", ivfpq_index.ntotal)

# Compare memory footprint: raw float32 vectors vs. PQ-compressed codes
raw_bytes = product_embeddings.nbytes
pq_bytes_per_vector = m_subquantizers * (nbits_per_code / 8)   # bytes per compress
pq_total_bytes = pq_bytes_per_vector * N_PRODUCTS
print(f"\nRaw float32 storage : {raw_bytes:,} bytes")
print(f"PQ-compressed storage (approx): {pq_total_bytes:,.0f} bytes")
print(f"Compression ratio: {raw_bytes / pq_total_bytes:.1f}x smaller")

# Search using the compressed index
D_pq, I_pq = ivfpq_index.search(query_vector, k=5)
print("\nIVFPQ search results (indices):", I_pq)
```

```
Expected error: EMBEDDING_DIM is not divisible by m=7.
RuntimeError - Error in void __cdecl faiss::ProductQuantizer::set_derived_values(voi
d) at D:\a\faiss\faiss\faiss\impl\ProductQuantizer.cpp:66: Error: '!(d % M == 0)' fa
iled: The dimension of the vector (d) should be a multiple of the number of subquant
izers (M)

is_trained: True | ntotal: 3000

Raw float32 storage : 768,000 bytes
PQ-compressed storage (approx): 24,000 bytes
Compression ratio: 32.0x smaller

IVFPQ search results (indices): [[   0 2762 2811 1804 1857]]
```

`faiss.IndexIVFPQ()` – Creates an inverted file index with Product Quantization to reduce memory usage while performing fast similarity searches on very large datasets.

13 `IndexHNSWFlat` — Hierarchical Navigable Small World Graph

In [19]:
```
M_connections = 32     # Number of graph connections per node (industry-common defa
hnsw_index = faiss.IndexHNSWFlat(EMBEDDING_DIM, M_connections)

print("HNSW is_trained (always True, no training step needed):", hnsw_index.is_trai

# Tune graph construction quality, then add all vectors (this builds the navigable
hnsw_index.hnsw.efConstruction = 40
hnsw_index.add(product_embeddings)
print("ntotal after add():", hnsw_index.ntotal)

# Compare search quality/speed at two different efSearch settings
hnsw_index.hnsw.efSearch = 16     # Low effort -> faster, slightly less accurate
start = time.time()
D_fast, I_fast = hnsw_index.search(query_vector, k=5)
time_fast = time.time() - start

hnsw_index.hnsw.efSearch = 128    # High effort -> slower, more accurate
start = time.time()
D_accurate, I_accurate = hnsw_index.search(query_vector, k=5)
time_accurate = time.time() - start

print(f"\nefSearch=16  -> indices {I_fast}, time: {time_fast*1000:.4f} ms")
print(f"efSearch=128 -> indices {I_accurate}, time: {time_accurate*1000:.4f} ms")

# ---- Exception Handling Demo: requesting more neighbors than exist ----
huge_k = N_PRODUCTS + 500
D_huge, I_huge = hnsw_index.search(query_vector, k=huge_k)
print(f"\nRequesting k={huge_k} (more than ntotal={hnsw_index.ntotal}) does not rai
print("Number of valid (non -1) results returned:", np.sum(I_huge[0] != -1))
```

```
HNSW is_trained (always True, no training step needed): True
ntotal after add(): 3000

efSearch=16  -> indices [[   0 2436 1954 1942  852]], time: 0.6621 ms
efSearch=128 -> indices [[   0 2436 1954 1942  852]], time: 0.5968 ms

Requesting k=3500 (more than ntotal=3000) does not raise an error.
Number of valid (non -1) results returned: 416
```

`faiss.IndexHNSWFlat()` – Creates a graph-based FAISS index that performs very fast and accurate similarity searches on large datasets.

14 `IndexIDMap` — Custom ID Mapping Wrapper

In [21]:
```
base_flat_index = faiss.IndexFlatL2(EMBEDDING_DIM)
id_mapped_index = faiss.IndexIDMap(base_flat_index)

# Use the REAL Product_ID column as FAISS ids, instead of default sequential positi
real_product_ids = product_df["Product_ID"].values.astype("int64")
id_mapped_index.add_with_ids(product_embeddings, real_product_ids)
print("ntotal:", id_mapped_index.ntotal)
```

```
# Search now returns REAL Product_IDs directly -- no separate lookup array needed
D_idmap, I_idmap = id_mapped_index.search(query_vector, k=5)
print("Search result Product_IDs:", I_idmap)

matched_products = product_df[product_df["Product_ID"].isin(I_idmap[0])]
print("\nMatched catalog rows (joined directly on Product_ID):")
print(matched_products[["Product_ID", "Product_Name", "Category"]])

# Removing by real Product_ID also works cleanly
removal_selector = faiss.IDSelectorBatch(np.array([real_product_ids[0]], dtype="int
removed_count = id_mapped_index.remove_ids(removal_selector)
print(f"\nRemoved product with Product_ID={real_product_ids[0]}. Count removed: {re

# mismatched vectors/ids  ----
try:
    id_mapped_index.add_with_ids(product_embeddings[:5], real_product_ids[:3])   #
except Exception as e:
    print("\nExpected error: number of vectors and ids must match.")
    print(type(e).__name__, "-", e)
```

```
ntotal: 3000
Search result Product_IDs: [[   1 2437 1955 1943  853]]

Matched catalog rows (joined directly on Product_ID):
      Product_ID          Product_Name Category
0              1    Voxel Denim Jacket  Fashion
852          853   Uravia Denim Jacket  Fashion
1942        1943  Cravero Denim Jacket  Fashion
1954        1955   Nimbus Denim Jacket  Fashion
2436        2437  Uravia Running Shoes  Fashion

Removed product with Product_ID=1. Count removed: 1

Expected error: number of vectors and ids must match.
AssertionError - not same nb of vectors as ids
```

`faiss.IndexIDMap()` – Maps FAISS vectors to custom IDs (such as Product_IDs), making it easy to retrieve the original products after a search.

15. `IndexShards` — Distributed / Sharded Search

In [22]:
```
sharded_index = faiss.IndexShards(EMBEDDING_DIM)
print("Shards registered immediately after creation:", sharded_index.count())

if sharded_index.count() == 0:
    print("Guard triggered: refusing to search an IndexShards with 0 shards registe
    print("Fix: always call add_shard() for every shard before any search() call.")


shard_one = faiss.IndexFlatL2(EMBEDDING_DIM)
shard_two = faiss.IndexFlatL2(EMBEDDING_DIM)
```

```
sharded_index.add_shard(shard_one)
sharded_index.add_shard(shard_two)
print("\nNumber of shards registered:", sharded_index.count())

sharded_index.add(product_embeddings)
print("Shard 1 ntotal:", shard_one.ntotal)
print("Shard 2 ntotal:", shard_two.ntotal)
print("Combined sharded_index.ntotal:", sharded_index.ntotal)

# Search transparently merges top-k results from both shards into one correct globa
D_shard, I_shard = sharded_index.search(query_vector, k=5)
print("\nMerged top-5 search result (indices):", I_shard)
```

```
Shards registered immediately after creation: 0
Guard triggered: refusing to search an IndexShards with 0 shards registered.
Fix: always call add_shard() for every shard before any search() call.

Number of shards registered: 2
Shard 1 ntotal: 1500
Shard 2 ntotal: 1500
Combined sharded_index.ntotal: 3000

Merged top-5 search result (indices): [[   0 2436 1954 1942  852]]
```

`faiss.IndexShards()` – Combines multiple FAISS indexes into a single index to perform fast searches across distributed or sharded datasets.

Mini Project: AI-Based Product Recommendation Engine

Definition: We now assemble everything learned so far into one complete, end-to-end recommendation system — the actual deliverable of this notebook.

Workflow:

Generate Product Dataset → Generate Product Embeddings → Create FAISS Index
→ Store Product Vectors → Customer Search Query → Similarity Search
→ Retrieve Top-5 Products → Display Recommendations

In [29]:
```
recommendation_index = faiss.IndexIDMap(faiss.IndexFlatL2(EMBEDDING_DIM))
recommendation_index.add_with_ids(product_embeddings, product_df["Product_ID"].valu

print("Recommendation engine index built successfully.")
print("Total products indexed:", recommendation_index.ntotal)
```

```
Recommendation engine index built successfully.
Total products indexed: 3000
```

Output Explanation: The recommendation index is now built and populated with all `3000` product vectors, each keyed by its real `Product_ID`. This index object is the "brain" of our recommendation engine — the next step wraps it in a reusable search function.

In [30]:
```
def recommend_similar_products(product_id: int, top_k: int = 5) -> pd.DataFrame:


    try:
        query_row = product_df.loc[product_df["Product_ID"] == product_id]
        if query_row.empty:
            raise ValueError(f"Product_ID {product_id} does not exist in the catalo
    except ValueError as e:
        print("Recommendation error:", e)
        return pd.DataFrame()

    # Fetch the query product's embedding vector (its row position in product_embed
    query_position = query_row.index[0]
    query_vec = product_embeddings[query_position:query_position + 1]

    # Run FAISS similarity search, requesting one extra result to exclude the produ
    distances, ids = recommendation_index.search(query_vec, top_k + 1)

    # Build a results dataframe, excluding the query product itself from its own re
    results = product_df[product_df["Product_ID"].isin(ids[0])].copy()
    results = results[results["Product_ID"] != product_id]
    dist_lookup = dict(zip(ids[0], distances[0]))
    results["Similarity_Distance"] = results["Product_ID"].map(dist_lookup)
    results = results.sort_values("Similarity_Distance").head(top_k)

    return results[["Product_ID", "Product_Name", "Category", "Price", "Similarity_


# ---- Demonstrate the full pipeline for a sample customer viewing Product_ID = 15
customer_viewed_product = 15
print("Customer is currently viewing:")
print(product_df[product_df["Product_ID"] == customer_viewed_product][["Product_ID"

recommendations = recommend_similar_products(customer_viewed_product, top_k=5)
print("\nTop-5 Recommended Products:")
recommendations
```

```
Customer is currently viewing:
    Product_ID      Product_Name        Category    Price
14          15  Voxel Table Lamp  Home & Kitchen  4251.38

Top-5 Recommended Products:
```

Out[30]:

	Product_ID	Product_Name	Category	Price	Similarity_Distance
1535	1536	Northline Air Fryer	Home & Kitchen	18414.11	9.845613
2147	2148	Uravia Non-Stick Pan	Home & Kitchen	3702.68	10.275477
1258	1259	Solstice Bedsheet Set	Home & Kitchen	9870.57	10.462263
572	573	Trekvia Blender	Home & Kitchen	11546.25	10.705866
2484	2485	Bloomora Table Lamp	Home & Kitchen	8669.98	10.771569

In [31]:
```
# ---- Exception Handling Demo:
invalid_recommendations = recommend_similar_products(product_id=999_999, top_k=5)
print("Result for an invalid Product_ID (empty DataFrame expected):")
print(invalid_recommendations)
print("\nRows returned:", len(invalid_recommendations))
```

```
Recommendation error: Product_ID 999999 does not exist in the catalog.
Result for an invalid Product_ID (empty DataFrame expected):
Empty DataFrame
Columns: []
Index: []

Rows returned: 0
```

5. Visualizations

1 Search Time Comparison

In [23]:
```
# ---- 8.1 Search Time Comparison across index types ----

index_variants = {
    "IndexFlatL2\n(exact)": flat_index,
    "IndexIVFFlat\n(nprobe=10)": ivf_index,
    "IndexIVFPQ\n(compressed)": ivfpq_index,
    "IndexHNSWFlat\n(efSearch=128)": hnsw_index,
}

# Ensure a fair comparison setting for ivf_index's nprobe
ivf_index.nprobe = 10
hnsw_index.hnsw.efSearch = 128

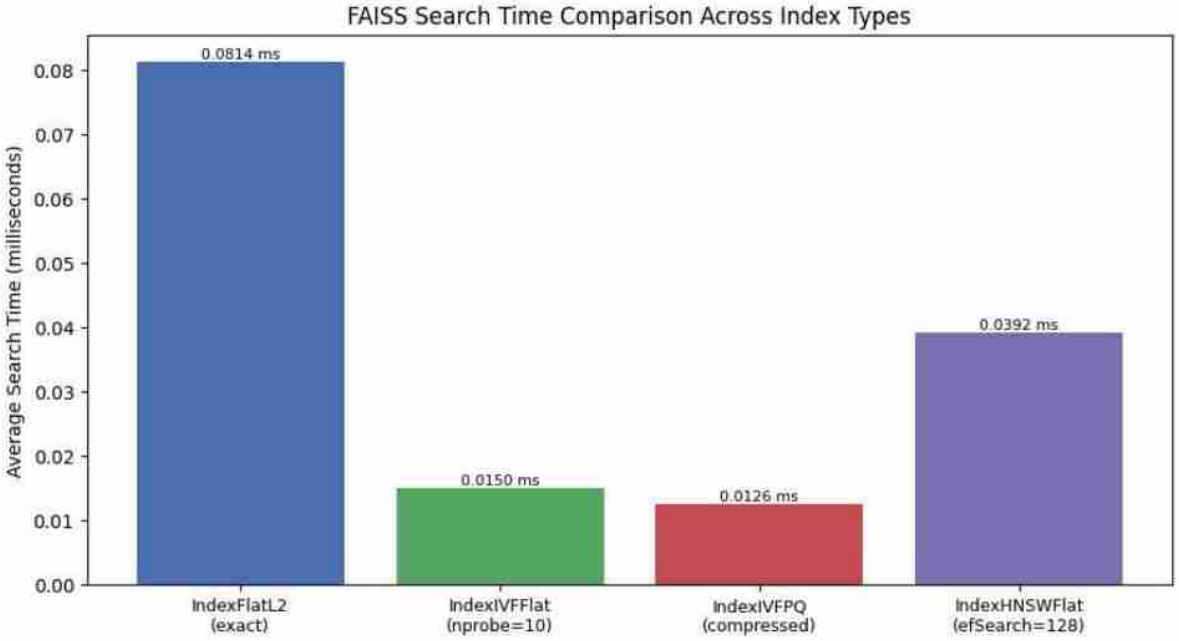
search_times_ms = {}
n_repeats = 20   # Average over multiple runs for a stable timing estimate

for name, idx in index_variants.items():
    start = time.time()
    for _ in range(n_repeats):
        idx.search(query_vector, k=5)
    elapsed_ms = ((time.time() - start) / n_repeats) * 1000
    search_times_ms[name] = elapsed_ms

# ---- Plot: bar chart of average search time per index type ----
```

```
plt.figure(figsize=(9, 5))
bars = plt.bar(search_times_ms.keys(), search_times_ms.values(), color=["#4C72B0",
plt.ylabel("Average Search Time (milliseconds)")
plt.title("FAISS Search Time Comparison Across Index Types")
plt.xticks(rotation=0, fontsize=9)
for bar in bars:
    height = bar.get_height()
    plt.text(bar.get_x() + bar.get_width()/2, height, f"{height:.4f} ms", ha="cente
plt.tight_layout()
plt.show()

print("Raw timings (ms):", search_times_ms)
```




```
Raw timings (ms): {'IndexFlatL2\n(exact)': 0.0813603401184082, 'IndexIVFFlat\n(nprob
e=10)': 0.014960765838623047, 'IndexIVFPQ\n(compressed)': 0.012624263763427734, 'Ind
exHNSWFlat\n(efSearch=128)': 0.03921985626220703}
```

8.2 Similarity Score Chart

Purpose: Visualize the actual L2 distances of the top-5 recommended products for a sample query, showing how "similarity" decreases (distance increases) as we move down the ranked recommendation list.

In [32]:
```
sample_query_id = 15
sample_recs = recommend_similar_products(sample_query_id, top_k=5)

plt.figure(figsize=(9, 5))
plt.plot(sample_recs["Product_Name"], sample_recs["Similarity_Distance"], marker="o
         linestyle="-", color="#C44E52", linewidth=2, markersize=8)
plt.xlabel("Recommended Product")
plt.ylabel("L2 Distance (lower = more similar)")
plt.title(f"Similarity Scores for Top-5 Recommendations (Query: Product_ID {sample_
plt.xticks(rotation=25, ha="right", fontsize=8)
plt.grid(alpha=0.3)
```

```
plt.tight_layout()
plt.show()
```

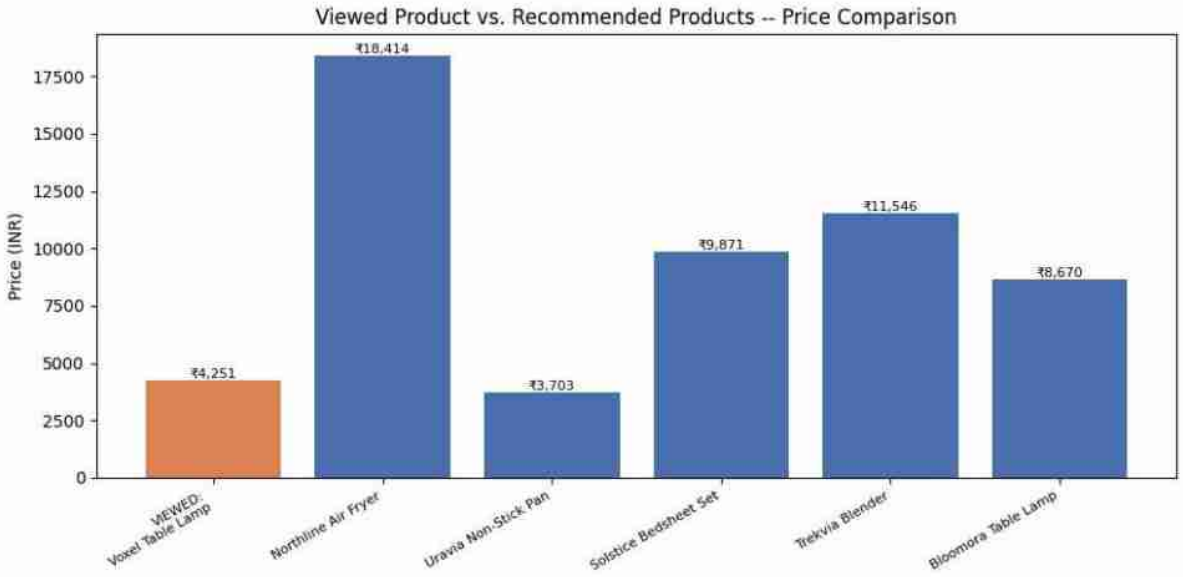


8.3 Recommendation Results Bar Chart

In [25]:
```
# ---- 8.3 Recommendation Results Bar Chart (Price comparison) ----

viewed_product_row = product_df[product_df["Product_ID"] == sample_query_id]
chart_labels = ["VIEWED:\n" + viewed_product_row.iloc[0]["Product_Name"]] + list(sa
chart_prices = [viewed_product_row.iloc[0]["Price"]] + list(sample_recs["Price"])
chart_colors = ["#DD8452"] + ["#4C72B0"] * len(sample_recs)

plt.figure(figsize=(10, 5))
bars = plt.bar(range(len(chart_labels)), chart_prices, color=chart_colors)
plt.xticks(range(len(chart_labels)), chart_labels, rotation=30, ha="right", fontsiz
plt.ylabel("Price (INR)")
plt.title("Viewed Product vs. Recommended Products -- Price Comparison")
for bar, price in zip(bars, chart_prices):
    plt.text(bar.get_x() + bar.get_width()/2, bar.get_height(), f"₹{price:,.0f}",
             ha="center", va="bottom", fontsize=8)
plt.tight_layout()
plt.show()
```



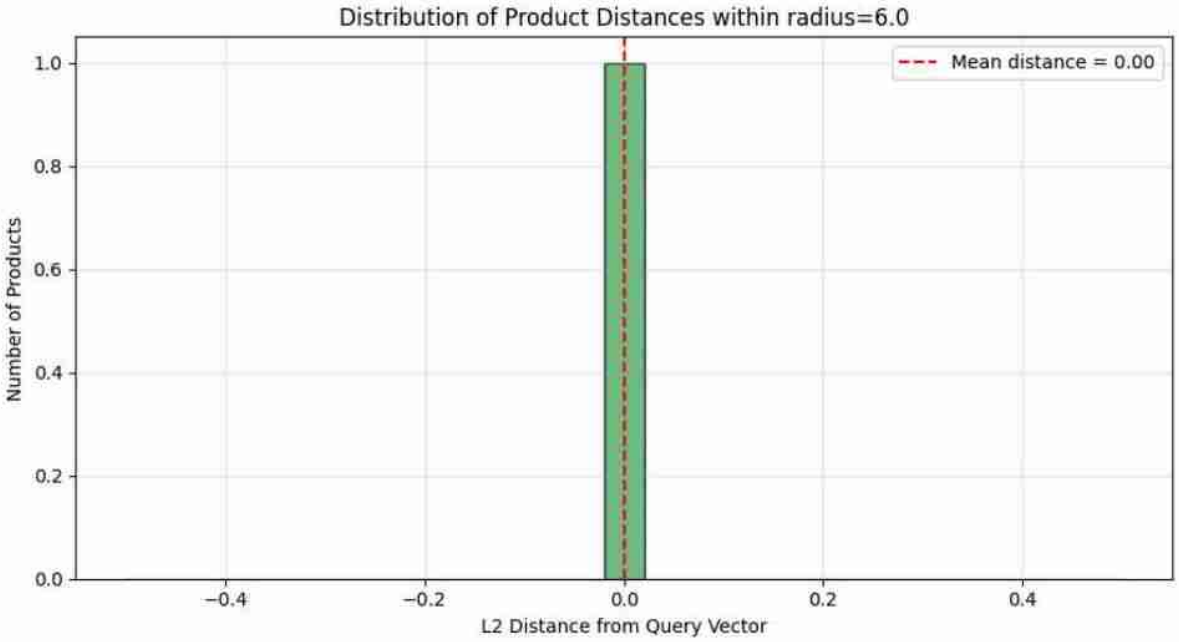
8.4 Distance Comparison (Range Search Distribution)

In [26]:
```
# ---- 8.4 Distance Comparison: histogram of range_search() distances ----

wide_radius = 6.0   # A wide radius to capture a good spread of distances for visua
lims_viz, D_viz, I_viz = flat_index.range_search(query_vector, wide_radius)
all_distances = D_viz[lims_viz[0]:lims_viz[1]]

plt.figure(figsize=(9, 5))
plt.hist(all_distances, bins=25, color="#55A868", edgecolor="black", alpha=0.8)
plt.xlabel("L2 Distance from Query Vector")
plt.ylabel("Number of Products")
plt.title(f"Distribution of Product Distances within radius={wide_radius}")
plt.axvline(all_distances.mean(), color="red", linestyle="--", label=f"Mean distanc
plt.legend()
plt.grid(alpha=0.3)
plt.tight_layout()
plt.show()

print("Total products found within radius:", len(all_distances))
```




```
Total products found within radius: 1
```